

Instruction formatting

Various addressing modes.

- * Immediate addressing $\rightarrow$ data inside the instruction.
: MOV R1, #10
- * Direct addressing $\leftrightarrow$ memory address is given: LOAD 500.
- * Indirect addressing $\rightarrow$ address points to another address.
LOAD (500)
- * Relative addressing $\rightarrow$ JUMP +500
- * Indexed addressing $\rightarrow$
base address + index register.
: LOAD 500(R1)

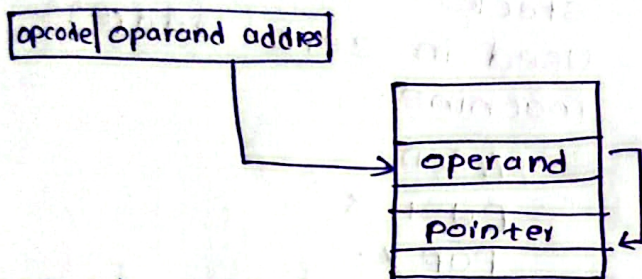
That mean's, inside of 500 * memory location there's another mem location.

$\rightarrow$
other
addressing
modes.

3) Indirect Addressing .

- * Instruction hold memory addresses , which in turns hold memory address of operand
- * Memory address. points to another address in the memory
- * Two memory access required
- * More space, as inside memory more space than instructions.

ADD R1, 20AH6



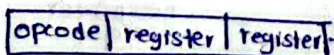
operand contains address of another memory location that is pointer.

4) Register Addressing .

- Operand held in registers, Instruction contains register's number.
- few bits needed , as register identified by 5 bits.
- Faster execution, as no memory access required to obtain an operand.

Eg : modern load store architecture.

ADD R1, R2, R3
R1 = R2 + R3

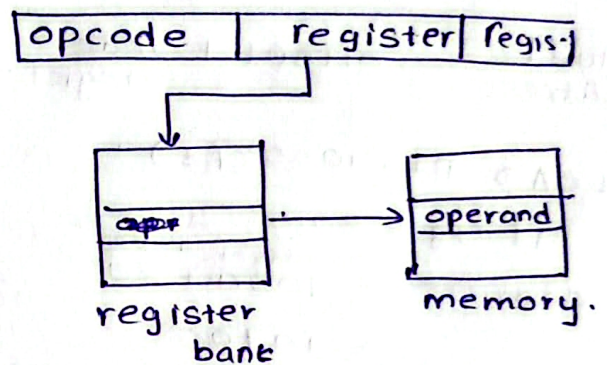


as) Register Indirect Addressing .

- * Instruction hold a register and register hold memory address . where operand stored .
- access large address space.

ADD R1, (R5) .

PC = R1 + Mem[R5]



Ob) Relative addressing .

Here instruction doesn't contain full memory address, it only contain small part of address called offset.

CPU calculates the address.

$$\text{actual Address} = \text{PC} + \text{offset}$$

If offset = 20 & -20

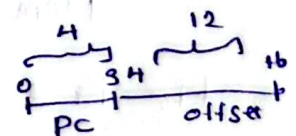
PC = 1000

If +20

A = 1020

If -20

A = 980



If offset has 12 bits .

$$2^{12} = 4096$$

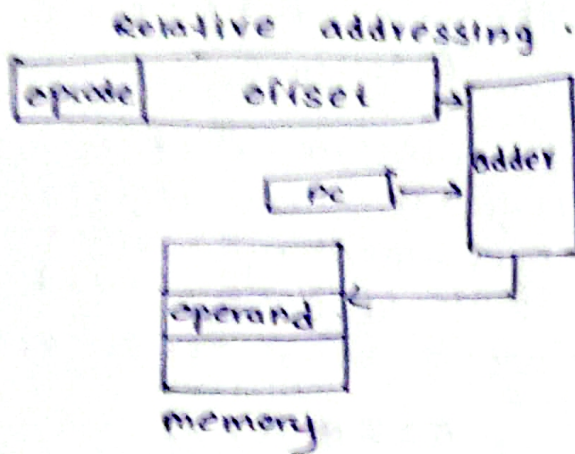
$$-2^{11} = -2048$$

$$2^{11} = 2048 - 1$$

offset range

$$-2048 \text{ to } 2047$$

values upto those ranges can added to offset



or) Indexed addressing.

Actual address = offset + Index register.

LOAD R1, 1050(R3).

if $R_3 = 20$.
 $\text{address} = \text{offset} + R_3$
 $= 1050 + 20$
 $= 1070$.

LOAD [1070]

LOAD Mem[1070].

Here special purpose register or general purpose register used as Index register.

Effective address obtained by adding offset to indexed register value.

① LOAD R1, 1050(R3)

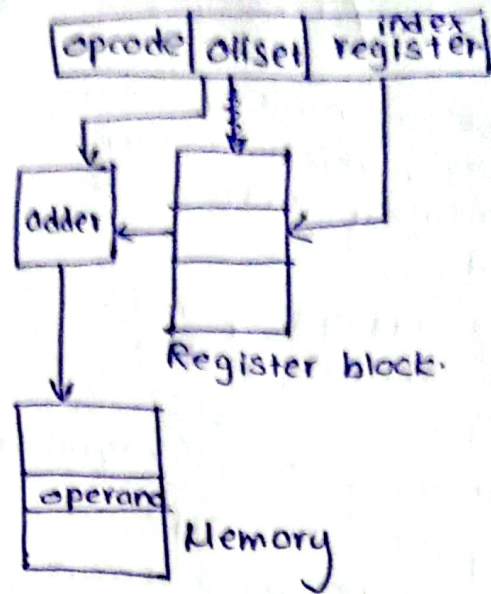
↑
Mem[1050 + R3]

$R_1 = R_1 + \text{Mem}[1050 + R_3]$.

② ADD R1, 1050(R3)

$R_1 = R_1 + \text{Mem}[1050 + (R_3)]$.

This sequentially access elements of array. offset gives starting & index reg represents array element to be used.



8) Stack Addressing.

- operand on the top of stack.
- Used in zero address machine.

ADD

PUSH x

POP y.

Many processors have special register called stack pointer to keep the track of stack - top of memory.

Other addressing methods.

① Base addressing,

processor has special register called base register which store address of block of memory.

Instruction itself contain offset.

Address = base register | offset.

Easy movement of code.

② Autoincrement & autodecrement

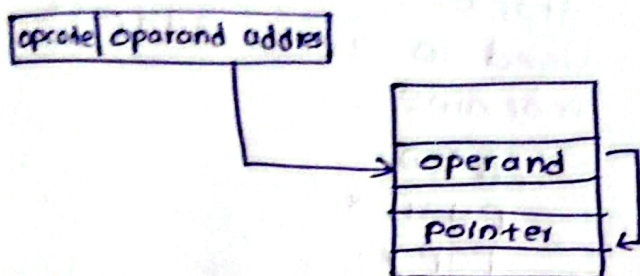
↑
increase register after using
a++

decrease register after using
a--

3) Indirect Addressing

- Instruction hold memory addresses, which in turns hold memory address of operand
- Memory address points to another address in the memory
- Two memory access required
- More space, as inside memory more space than instructions.

ADD R1, 20AH6



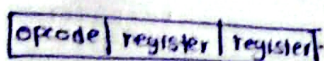
operand contains address of another memory location that is pointer.

4) Register Addressing

- Operand held in registers, Instruction contains register's number.
- few bits needed, as register identified by 5 bits.
- Faster execution, as no memory access required to obtain an operand.

Eg: modern load, store architecture.

ADD R1, R2, R3
R1 = R2 + R3

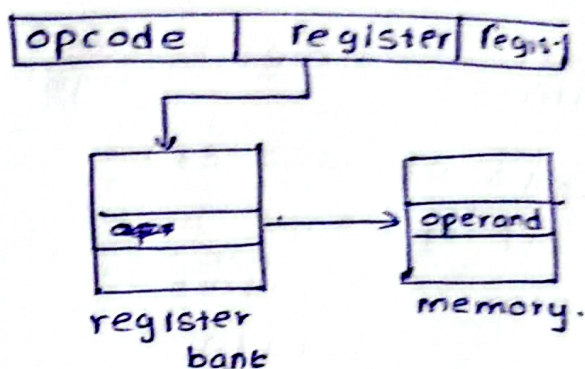


as) Register Indirect Addressing

- Instruction hold a register and register hold memory address. Where operand stored.
- access large address space

ADD R1, (R5)

PC = R1 + Mem[R5]



ob) Relative addressing

Here instruction doesn't contain full memory address, it only contain small part of address called offset.

CPU calculates the address.
actual
Address = PC + offset

If offset = 20 & -20

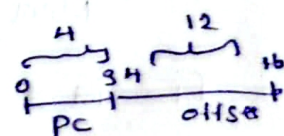
PC = 1000

If +20

A = 1020

If -20

A = 980



If offset has 12 bits.

$2^{12} = 4096$

$2^{11} = -2048$

$2^{11} = 2048 - 1$

offset range

= 2048

values upto those ranges can added to offset

CISC & RISC architecture

Features that used for mainframe computers & super computers are later used for new generation microprocessors.

old computers.

memory was expensive

programmes written on assembly languages. Then engineers

tried to reduce number of instructions on program

by creating powerful, complex instructions. This is

CISC.

Later

memory become cheap:

CPU become faster. Then

engineers found many

simple instructions could

execute program effectively.

This is RISC.

CISC - Complex instructions Set computer

RISC - Reduced instructions Set computer.

CISC

- powerful complex instructions.
- One instruction perform several tasks.
- Goal is to reduce number of instructions by creating a complex instruction.
- large number of addressing modes.
- Special purpose registers & flags (sign, zero, carry).
- variable lengths. (not all have same size)
- Easily map to high level languages.
- Complex instruction decoding.
- pipeline implementation is complex.

pipeline allows several instructions to ~~perform~~ process at same time in different stages.

Eg: IBM 360/370.

VAX 11/780

Intel x86 ← used for more generations.

as volume of chip manufactured is so high
~~reason that is enough motivation to pay extra design cost.~~

- backward compatibility.
old SW works for new computers.
- ^{if hos} sufficient hardware to translate CISC to RISC internally.

മിക്ക CISC വാൾ സൂപ്പർ മൈക്രോ
 RISC ലെ convert ആണ് ചെയ്യുന്നത്.

RISC

- Load store instructions access memory.
- All instructions process on registers.
- Easy for pipeline
- few addressing mode as simple instructions set.
- Large number of general purpose registers, very few special purpose registers
- Instruction length is small. So easy to decode.
- As pipeline improved, performance high.

Eg: CDC 6600.

SPARC.

- Use compilers to translate RISC instructions
- CISC use hardware to translate ~~RISC~~ into RISC instructions.

in x86, there's a separate flag register to store the status of bits.

VAX 8700 machine [CISC machine]

- fewer instructions.

* RISC processors use simple instructions. so it has many number of instructions to perform the task.

^{clock}
- Cycles per instruction 6 times greater than MIPS.

as CISC is more complex. it need to do more work for one instruction. so it required more clock cycles.

- low performance.

as RISC has more simple instructions. it perform well & it's easy for pipeline. & it need lower ^{clock} cycle per instruction.

- more hardware

In CISC it has more hardware resources. to translate into RISC.

Conclusion

- Expensive because of more hardware.
- Usage of more addressing modes
- Complex instructions take more clock cycle.
- Difficulty for pipeline implementation.

So traditional CISC become inefficient.

Vax was replaced by ALPHA (a RISC processor) by DEC.

after this,

- high performance
- simple hardware design.
- better pipeline
- low complexity.

MIPS M2000 [RISC machine]

- More instructions.

^{clock}
- Lower cycles per instruction.

- performance is high (x3)

- less hardware

After CISC architectures, Intel x86 survived, because it has huge installed base (large amount of SW was written) & backward compatibility.

* x86 adopted a balance approach. (they combined both advantages of RISC and CISC).

(a) User view CISC instruction set

(b) Hardware translate every CISC to RISC

(c) Instruction pipeline execute RISC instructions efficiently.

MIPS32 architecture

- Instruction set of MIPS32

* This defines all the operations that processor can perform. MIPS32 has small set of instructions.

- Encoding details of MIPS32

* Define how instructions represent in binary form (use 32 bits).

- Design the data path & look into CU design issues

data path perform data processing operations. (ALU, registers, memory, ...)
CU generate control signals to operate data path.

- Extend data path to pipeline architecture.

MIPS CPO registers

* GPRs

- MIPS32 has 32 registers R0 to R31

- each register has 32 bits.
- They store data & addresses.

* PC

- A special 32 bit register
- Store addresses of next instructions.
- Instructions like J (Jump)
eg: $PC + 4 \leftarrow PC$
- No direct visible to programmer.

* HI & LO registers

- Use for multiplication and division.

multiplication.

HI - upper 32 bits.

LO - lower 32 bits.

Div

HI - remainder

LO - quotient.

* Stack pointer register (SP)

- any GPR can be used as SP
- To maintain stack in the memory.
- No separate PUSH, POP, CALL instruction.

* Index Register (IX)

- Any GPR.
- To access memory words sequentially.

* Flag Registers

- ZERO, SIGN, CARRY.
Keep track of results of arithmetic & logic operations.
- To avoid problems in pipeline.